

Buddiz (YAEL) - Diagnostic complet avant mise en production

28 se

1. C'est quoi ?

- Marketplace de repas partagés (France, UI français, EUR, Europe/Paris). Ni livraison, ni réservation restaurant, ni dating.
- Un compte = hôte + invité.
- Cycle : Découverte > Messagerie > Demande (REQUESTED) > Acceptation hôte (ACCEPTED_PENDING_PAYMENT) > Paiement (CONFIRMED) > Révélation adresse exacte > Repas (COMPLETED) > Avis.
- Fonctions : auth email/password + Google + bouton Apple, profil + avatar, feed Découvrir + recherche live + filtres (distance, catégories, intérêts), publication / édition / retrait d'annonce, messagerie participants-only (polling), notifications in-app, avis post-repas, annulation avec remboursement ($\geq 24h$ 100%, $< 24h$ 50%), revenus hôte.

2. Comment c'est fait ?

- Frontend : Expo 57.0.19 / React Native 0.86.3 / React 19.2.3 / Expo Router (file-based, typedRoutes:true) / TanStack Query 5.102.8 / expo-secure-store / Inter / Feather. Routes : app/index.tsx (gate), (auth)/welcome,login,register, (tabs)/index=Decouvrir,add,messages,profile, meal/[id], conversation/[id], payment/[bookingId], cancel/[bookingId], review/[bookingId], filters (modal), notifications, earnings, connect-return, edit-profile, feedback, assistant. Etat : AuthProvider + FiltersProvider + ToastProvider + QueryClient singleton. Client API : src/api/client.ts (fetch BASE/api + Bearer).
- Backend : FastAPI 0.110.1 + Starlette 0.37.2 + Uvicorn 0.25.0 + Pydantic 2.13.5, monolithe backend/server.py (1854 lignes), APIRouter prefix /api. 49 routes : auth x5, users x2, upload/files/config x4, meals x7, conversations/messages x4, bookings x7, payments x4, earnings/connect x6, webhook x1, address/cancel/reviews/feedback/notifs/AI x9.
- DB : MongoDB via Motor 3.3.1 + PyMongo 4.6.3, 12 collections implicites sans ODM (users, user_sessions, meals, bookings, conversations, messages, notifications, reviews, uploads, app_config, app_feedback, stripe_events). Montants en cents, fee 10% configurable.
- Auth : JWT HS256 TTL 30j + user_sessions pour Google. Upload via Emergent Objstore. Paiements : Stripe 14.4.1 + Connect Express + Checkout (destination_charge, application_fee).
- Tests : backend/tests/ (4 fichiers), pytest -n 2 --dist loadscope. Rapports iteration_1/2/3 annoncent 23/23 + 15/15 + 9/9. Aucun test frontend versionné. Infra actuelle : image preview Emergent uniquement.

3. Blocages critiques avant production

- B1. Secrets sur disque (backend/.env, frontend/.env, frontend/app/.env) : JWT, clés Stripe test, clé LLM en clair, webhook vide. Non versionnées (vérifie git ls-files) mais rotation + vault (Doppler/Infisical) obligatoires.

- B2. Webhook Stripe inoperant (server.py:1357-1360, raise 400 si secret vide). Sans whsec, pas de confirmation fiable.
- B3. CORS ouvert (server.py:1843-1849) : allow_origins=[*] + allow_credentials=True -> incoherent navigateur + CSRF. Restreindre aux domaines prod.
- B4. Seed auto au startup (server.py:1831,1665-1805) : recree users demo + repas + bookings en prod. Gater par ENV!=production.
- B5. URLs preview en dur (APP_PUBLIC_URL, EXPO_PUBLIC_BACKEND_URL) : cassent success_url/cancel_url (server.py:1181-1182) et connect return/refresh (server.py:1307-1354).
- B6. Aucun artefact deploiement : pas de Dockerfile, docker-compose, CI (.github/workflows), eas.json, app.config.js. Bundle com.emergent.localfeast... temporaire (app.json:12,24).
- B7. RGPD/stores bloquant : pas de DELETE /users/me, export, CGU/confidentialite, moderation/signalement. email_verified jamais verifie (server.py:389).
- B8. Auth fragile : logout ne revoque pas le JWT (server.py:474-479), TTL 30j (server.py:41), pas de rate-limit sur register/login/session, password min 6.
- B9. Concurrency capacite : TOCTOU create_booking / update_guest_count (server.py:985-1000,1073-1075) sans transaction atomique -> surbooking possible.
- B10. Upload sans limite taille ni verif magic-bytes, content_type client trusted (server.py:506-528). GET /files/{path} public (server.py:532-541).
- B11. Pas de pagination (to_list 500/1000/2000) + tri memoire + N+1 massif (meal_public, host_earnings, host_reviews).
- B12. connect/refresh/{account_id} sans auth ni ownership (server.py:1351-1354).
- B13. Crash au boot si env absent (os.environ[] server.py:35,39).
- B14. Frontend : BASE="" sans validation (client.ts:5), AUTH_BASE en dur (oauth.ts:11), LogBox.ignoreAllLogs (app/_layout.tsx:19), token web en clair (storage/index.web.ts:49), paiement en boucle 8x verify sans deep-link (payment/[bookingId].tsx:60-71).
- B15. Pas de monitoring/backup : logging basique (server.py:64), pas de Sentry/health/metrics, pas de PITR.

4. Problemes majeurs

- M1. Validation Pydantic sans bornes : MealCreate/MealUpdate (titre/description/prix/max sans max, status libre), messages/avis/feedback sans limite, ProfileUpdate sans validation URL, regex city injectable (server.py:586, sans re.escape).
- M2. Paiements non idempotents : double increment guests_welcomed_count, refund_cents non persiste en test, checkout sans try, erreurs Stripe fuient au client.
- M3. Index manquants : pas d'unique sur bookings.id / conversations.id, pas d'index messages.conversation_id, notifications.user_id, meals.city/starts_at.
- M4. Frontend : catch silencieux (AuthContext), QueryClient sans defaults, useQuery sans isError (sauf 2 ecrans), pas d'intercepteur 401, ~25 any + @ts-nocheck, TS 6.0 non-standard, react-native-dotenv mort, double date-fns+dayjs, polling 2-8s partout.
- M5. requirements.txt bloat : boto3/pandas/numpy/google/black/mypy/litellm via URL privee sans hash.

- M6. Tests live vers preview sans mocks, 2 tests faux (access_token vs token, payload inconnu) ; bcrypt sync bloquant ; startup/shutdown deprecie ; COMPLETED paresseux sans cron.

5. Check-list mise en production

- 1) Vault + rotation cles, gitleaks, .env.example.
- 2) Stripe : cles prod/test, webhook whsec, Connect Express active, test E2E + remboursement reverse_transfer.
- 3) Dockerfile + compose mongo/api, CI pytest+tsc+eslint+expo-doctor, eas.json, bundle ID prod, versionCode/buildNumber.
- 4) CORS prod, slowapi (auth/upload), JWT court + refresh + blacklist, verif email, DELETE /users/me + pages legales.
- 5) Validation : re.escape, Field(max_length/ge/le), limite upload + verif Pillow, pagination skip/limit, transaction capacite, auth connect/refresh, index Mongo, idempotence paiements, seed gate, URLs prod.
- 6) Observabilite : Sentry, /api/health avec ping DB, logs JSON, backup Atlas PITR + runbook restore.
- 7) Frontend : validation BASE, 401 auto, QueryClient defaults, gestion isError partout, deep-link paiement, retirer LogBox.ignoreAllLogs, split server.py en routers.